

编 号: _
审定成绩: _____

重庆邮电大学

本科毕业设计（论文）



中文题目	基于移动平台的
	固定目标识别及瞄准系统设计
英文题目	Fixed-Target Recognition and
	Aiming System on a Mobile Platform
学院名称	自动化/工业物联网学院
学生姓名	杨应添
专 业	自动化
班 级	08902201
学 号	2022213347
指导教师	蔡军教授
答辩人	姓名职称

2026 年 6 月

重庆邮电大学教务处制

学院本科毕业设计(论文)诚信承诺书

本人郑重承诺：

我向学院呈交的论文《 》，是本人在指导教师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明并致谢。本人完全意识到本声明的法律结果由本人承担。

年级

专业

班级

承诺人签名

年 月 日

学位论文版权使用授权书

本人完全了解重庆邮电大学有权保留、使用学位论文纸质版和电子版的规定，即学校有权向国家有关部门或机构送交论文，允许论文被查阅和借阅等。本人授权重庆邮电大学可以公布本学位论文的全部或部分内容，可编入有关数据库或信息系统进行检索、分析或评价，可以采用影印、缩印、扫描或拷贝等复制手段保存、汇编本学位论文。

(注：保密的学位论文在解密后适用本授权书。)

学生签名:

指导老师签名:

日期: 年 月 日

日期: 年 月 日

摘要

无人化装备的智能化、低成本化趋势日益明显，如何在移动平台上实时识别并精确瞄准固定目标，已成为当前研究的热点方向。然而，已有方案要么成本过高，要么部署繁琐，实时性也常常差强人意。针对这一问题，本工作设计了一套面向移动平台的固定目标识别与瞄准系统，核心思路定位于“千元级成本”和“纯视觉+IMU紧耦合”，力求给出一个工程上可复制、可落地的低造价方案。

整个系统搭建为三节点分布式异构架构：一个 Linux 视觉节点专职做目标检测、PnP 位姿求解和跟踪状态管理；一块 STM32H743 作为云台主控，以 480 MHz 主频跑 RT-Thread 系统，负责双闭环姿态控制和 IMU 前馈补偿；另有一块 MSPM0G3507 独立管理底盘的八路灰度循迹和差速编码器 PI 速度闭环。三个节点之间通过串口协议交换信息，各自运行互不干扰。

视觉方面，识别和跟踪由三级筛选流程完成——HSV 颜色阈值分割、轮廓几何约束、PnP 重投影尺寸校验，可对 A4 白纸目标做出可靠的检测和中心定位。配合三线程加双有界队列的结构，视觉吞吐稳定维持在 25 FPS 以上。

云台控制端，设计了一个双闭环结构：外层是像素误差驱动的 APID，内层是 IMU 角速度和加速度前馈，前馈部分的思路参考了自抗扰控制。在平台突然加速或受到振动冲击时，这组结构能把相位滞后明显压低。实测显示，瞄准线的峰值偏差因此降低了 30% 以上，稳态指向误差也落到了 $\leq 0.8 \text{ mrad}$ 的指标以内。所用的 HiPNUC 六轴 IMU 以 100 Hz 输出角速度和姿态数据，经 hipnuc_dec 协议模块完成字节流解析和 CRC 校验。

底盘一侧，通过差速运动学模型做速度分解，左右轮各自走增量式 PI 调速加 AB 相编码器反馈闭环。循迹策略把归一化灰度加权偏差估计和一阶滞后低通滤波结合起来，传感噪声引起的控制抖动得到了有效抑制。此外还集成了 A* 启发式路径规划，使得小车在结构化场地里能完成点对点自主移动。

实验数据表明，室内标准光照下视觉帧率稳定超过 25 FPS，IMU 采样保持在 100 Hz，底盘可以稳定完成多圈循迹。整机的器件成本控制在千元量级，验证了这套方案具备工程可行性和实用价值。

关键词：视觉伺服，目标识别，云台控制，IMU 前馈，移动机器人

Abstract

With the growing demand for low-cost intelligent unmanned systems, real-time recognition and accurate aiming at fixed targets on mobile platforms has become a prominent research topic. Existing solutions are typically hindered by high cost, deployment complexity, or insufficient real-time performance. This thesis proposes and implements a fixed-target recognition and aiming system for mobile platforms, designed under the constraints of less than 1000 RMB and “pure vision + IMU tight coupling,” providing an engineering-practical low-cost solution.

The system adopts a three-node distributed heterogeneous architecture. The Linux vision node handles target detection, PnP pose estimation, and tracking state management. The STM32H743 gimbal controller, running the RT-Thread real-time operating system at 480 MHz, executes a dual-loop attitude control algorithm with IMU feedforward compensation. The MSPM0G3507 chassis controller independently manages eight-sensor grayscale line tracking and differential-drive encoder PI speed control. The three nodes communicate via serial protocols and operate without mutual interference.

For visual detection and tracking, a three-stage filtering strategy is adopted: HSV color threshold segmentation, contour geometric constraint verification, and PnP reprojection size validation. This enables robust detection and center localization of A4-paper targets. A three-thread plus dual-bounded queue architecture ensures stable visual throughput above 25 FPS.

For gimbal control, a dual-loop architecture is proposed: an outer APID loop driven by pixel error, and an inner feedforward loop driven by IMU angular velocity and acceleration. The feedforward design is inspired by Active Disturbance Rejection Control (ADRC). Under platform acceleration or vibration disturbances, the architecture significantly reduces phase lag, achieving a reduction of more than 30% reduction in peak aiming-line error in experiments. The steady-state pointing error satisfies the design target of ≤ 0.8 mrad. The HiPNUC six-axis IMU provides 100 Hz angular velocity and attitude output, decoded via the `hipnuc_dec` protocol module.

For chassis control, a differential kinematics model decomposes forward and angular velocity commands into individual wheel speed targets. Each wheel employs an incremental PI speed control with AB-phase encoder feedback. The line-following strategy combines normalized grayscale weighted deviation estimation with a first-order low-pass filter to suppress control jitter. Additionally, an A* heuristic path planning algorithm is integrated for autonomous point-to-point maneuvering in structured environments.

Experimental results demonstrate that the visual frame rate stably exceeds 25 FPS under standard indoor lighting, the IMU maintains 100 Hz sampling, and the chassis completes multi-lap line-following tasks reliably. The total hardware cost remains within the budget, validating the feasibility and engineering value of the proposed system.

KEY WORDS: Visual Servo, Target Recognition, Gimbal Control, IMU Feedforward, Mobile Robot

目录

第 1 章	引言	1
1.1	研究背景	1
1.1.1	应用背景	1
1.1.2	技术背景	1
1.2	研究目的及意义	2
1.2.1	研究目的	2
1.2.2	研究意义	2
1.3	研究现状及设计思路	2
1.3.1	国外研究现状	2
1.3.2	国内研究现状	3
1.3.3	存在的主要问题	3
1.3.4	本文的设计思路	3
1.4	发展趋势	4
1.5	主要内容和论文组织	4
第 2 章	系统总体设计	5
2.1	系统设计工作原理及原理框图	5
2.1.1	基本工作原理	5
2.1.2	系统原理框图与模块说明	5
2.2	系统设计要求	5
2.2.1	功能性要求	5
2.2.2	性能指标要求	6
2.2.3	约束条件与设计边界	6
2.3	关键器件选型与接口设计	6
2.3.1	主控器件	7
2.3.2	传感器与视觉模块	7
2.3.3	执行机构与接口设计	7
2.4	本章小结	8
第 3 章	系统硬件设计	9
3.1	单片机控制核心电路	9
3.2	IMU 数据采集电路	9
3.2.1	IMU 内部结构与工作模式	9
3.2.2	IMU 时序与采样配置	10
3.2.3	IMU 接口连接与抗干扰设计	10
3.3	图像采集与通信电路	10
3.3.1	USB 摄像头接入原理	10
3.3.2	图像数据链路设计	12
3.4	云台驱动与电机控制电路	12
3.4.1	电机驱动原理	12
3.4.2	RS485 总线接口与电源设计	12
3.5	自动循迹底盘控制电路	14

3.6	系统硬件联调与整机连接图	14
3.7	本章小结	14
第 4 章	系统软件设计	16
4.1	软件总体架构与流程	16
4.1.1	程序分层与任务划分	16
4.1.2	软件主流程图	16
4.2	IMU 数据处理程序	17
4.2.1	数据采集与滤波	17
4.2.2	姿态解算与角速度发布	17
4.3	视觉识别与跟踪程序	18
4.3.1	视觉线程架构与数据流	18
4.3.2	目标检测与几何筛选流程	19
4.3.3	PnP 位姿解算与目标量生成	20
4.3.4	丢失重捕获与串口通信协议	20
4.4	双闭环控制程序设计	21
4.4.1	像素误差外环 PID	21
4.4.2	IMU 角速度前馈内环	21
4.5	通信与上位机控制程序	22
4.5.1	USB-CDC 通信协议	22
4.5.2	Linux 端控制界面与参数下发	22
4.6	系统主控程序设计与异常处理	22
4.7	差速底盘工程的软件实现	23
4.7.1	控制主循环与运行模式	23
4.7.2	红外循迹与路口判别策略	24
4.7.3	固定圈速实现与圈数管理	24
4.7.4	差速运动学与轮速闭环	24
4.7.5	作为系统子模块的协同与可靠性优势	25
4.8	本章小结	25
第 5 章	系统调试与结果分析	26
5.1	调试环境与测试方案	26
5.2	图像识别与跟踪效果测试	26
5.3	姿态采集与补偿性能测试	26
5.4	云台控制与稳态误差测试	26
5.5	底盘循迹与圈速测试	26
5.6	整机联调结果与问题分析	27
5.7	本章小结	27
第 6 章	总结与展望	28
6.1	主要工作与创新点	28
6.2	不足与改进方向	28
6.3	后续研究工作展望	28
参考文献		31
致谢		33
附录 A	科技写作中非学术型低级错误的主要表现	34

附录 B 英文翻译	36
-----------------	----

第 1 章 引言

1.1 研究背景

1.1.1 应用背景

过去三十年里，无人化装备从最初的“遥控式”一步步向更自主的方向进化，移动平台上如何高精度地识别和瞄准固定目标，也自然成了工程和研究关注的焦点。军事需求是最早的驱动力：上世纪 90 年代，美军在“猎人”无人机上测试光电瞄准系统，开创了以视觉为核心的“平台动、目标静”瞄准模式；到了 2010 年以后，北约和主流军工企业把“视觉伺服加惯性稳定”写进了移动平台火控的设计准则，这个方向才算真正从实验室走向了工程实用^[1]。

国防之外，民用和商用场景同样迫切需要轻便、便宜又好部署的移动瞄准与跟踪方案。安防监控、交通监测、无人机巡检、精细农业（比如病虫害定点喷洒）、影视拍摄等，哪一样都要求在有限功耗和重量下实现够用的指向精度和鲁棒的跟踪。拿城乡安防来说，街道上的移动巡检车要在各种光照和遮挡条件里持续盯住目标；农业喷洒场景下，设备成本低、续航长是硬约束，根本用不起大功耗传感器阵列，轻量级的视觉加惯导方案自然成了首选。

落到工程实现上，这类系统躲不开一堆约束：传感器和处理器必须在很小的尺寸、重量和功耗预算里协同工作；时间同步和通信延迟直接卡着闭环控制的脖子；复杂环境下还得扛住遮挡、光照变化和多目标干扰。正是这些现实困难，让近几年的研究不再只盯着检测精度，而是更多关注系统级的时延、能耗和可复现性，也给面向教学和工程验证的低成本方案提供了存在的理由。

1.1.2 技术背景

从技术角度看，移动平台瞄准系统要跑起来，离不开视觉传感、惯性测量、实时嵌入式计算和机电稳定（云台）这几个方向的交叉。国内虽然起步晚一些，但 2015 年以来陆续有几款车载光电转台投入使用，说明在小型化陀螺稳定和实时图像跟踪上已经有了工程级的突破。平时衡量一套轻量化方案行不行，常用“千元成本、5 W 功耗、500 g 重量”这类指标——也恰恰是这些约束在推动学术研究往工程实现上靠。

几个关键技术维度值得展开说。第一是传感器怎么选：工业相机、激光雷达、民用 USB 摄像头、低成本六轴 IMU……选什么直接决定了精度、体积、功耗和成本之间的取舍。第二是实时处理和算法：传统方法比如 HOG+SVM 在资源受限场景下性价比不错，而轻量化神经网络（MobileNet、Tiny-YOLO 这类）最近几年靠模型压缩和量化，也慢慢能在嵌入式平台上跑出可用帧率了。第三是时间同步：高频 IMU 和低频视觉要做紧耦合，精确时间戳和低延迟通信是前提，否则前馈补偿和闭环控制的效果会大打折扣。第四是机械和控制：云台的机械结构好不好、减震和电机驱动策略到不到位，最后都反映在指向精度上。

软件架构上，嵌入式平台（比如 STM32H7）配合上位机（Linux 视觉端）的分工方式已经比较普遍了：上位机干视觉推理和图像处理的重活，主控板管 IMU 采样、闭环控制和实时执行，这样既保证了性能，又不至于把主控端算力撑爆。常见的工程做法还包括用 DMA 和中断来保证 IMU 采样的高精度，走 RS485 这类差分总线跟云台电机可靠通信，以及在控制环路里加前馈和预测补偿来弥补视觉反馈滞后的亏。

总的来看，技术背景有两条并行的线索：一条是追求极致精度，还是靠高端传感器和大算力平台；另一条是为了满足成本和功耗约束，把研究重心往算法和系统协同优化

上挪——比如轻量模型、传感器融合、时序对齐这些方向。后一条路恰恰给本科和工程验证级别的研究留出了可行的空间。

1.2 研究目的及意义

1.2.1 研究目的

本课题设计并实现基于移动平台的固定目标识别及瞄准系统，采用“纯视觉+IMU”紧耦合的轻量化方案，在满足成本约束的前提下，实现以下具体目标：

1. 构建硬件平台与通信基础：搭建以 STM32H7 为核心的小型化云台系统，集成 USB 摄像头与六轴 IMU，实现云台的实时远程控制；
2. 实现环境感知与角速度采集：完成高频（ ≥ 100 Hz）运动基座状态信息采集与处理；
3. 完成视觉跟踪与像素闭环：实现 25 FPS 连续跟踪，瞄准线稳态误差控制在 0.8 mrad 以内；
4. 集成 IMU 前馈与振动抑制：在像素闭环基础上引入 IMU 补偿，对 10~200 Hz 频段进行实时补偿。

1.2.2 研究意义

这项工作学术层面填补了国内在轻量化动基座瞄准方案上的一些空白，在应用层面则为无人系统提供了一条可复现、低成本的参考路径。搭起“纯视觉+低成本”这个设计框架之后，未来如果想接入激光雷达或者事件相机，硬件和算法的接口也都预留好了。

1.3 研究现状及设计思路

1.3.1 国外研究现状

国外在这方面起步早、投入大，基本上走的是“高精度、高成本、重平台”的路子。传感器方案上，激光雷达+视觉+惯导的三元融合几乎成了标配。

举个例子，卡内基梅隆大学 2022 年公开的实验里，他们的无人地面车辆装了 128 线激光雷达、工业级可见光相机（ $1920 \times 1440 @ 30$ Hz）和光纤陀螺，组成了一套三位一体的传感器系统。靠 Transformer 检测头^[2]做到 30 Hz 实时识别，能在复杂城区环境里精确定位固定目标，像素误差控制在 2 个像素以内，折合角度精度大约 0.3 mrad。但问题也很明显：整套系统成本超过 2 万美元，功耗 45 W，对移动平台的续航是个很大的拖累。这种方案放到本科教学层面，想做到“低成本、易复现”基本不现实。

德国的弗劳恩霍夫应用研究所（Fraunhofer IAO）走得更远，把 FPGA 和 GPU 的异构计算架构引进了瞄准闭环。GPU 上跑 Faster R-CNN 做目标检测，FPGA 上并行执行 IMU 前馈补偿，利用异构计算的高吞吐把定位误差压到了 1 cm 量级（对应 0.05 mrad 的角误差）。但代价也不小：系统重 6 kg，供电得 24 V、10 A，单是陀螺稳定分系统的成本就超过 5000 元，想在 5 kg 以下的微型无人车上部署，几乎不可能。

传感器层面，国外普遍用工业级高精度 IMU（漂移率 $< 1^\circ/\text{h}$ ）和光纤陀螺（精度 $< 0.01^\circ/\text{s}$ ），比民用级传感器贵出一个数量级还不止。Xsens 的 MTi-630 AHRS 要 8000 元以上，光纤陀螺类产品更是数万元起步。靠堆传感器换取高精度，代价实在太大了。

总的来看，国外研究强调“多传感器融合+大算力换高精度”，靠物理层的传感器冗余和算法层的计算冗余把性能指标推到极致。精度上确实遥遥领先^[3]，但这种技术路线对资源受限的场景——比如微型无人车、教学实验平台——天然就水土不服。

1.3.2 国内研究现状

受成本约束影响，国内研究走的路子要务实得多。大多数团队选择的是“纯视觉加惯导”的轻量化路线^[4]，核心理念是：硬件配置不够，算法设计来凑——在有限的计算资源和传感器精度下，仍然要让瞄准精度达到工程可用的水平。

早期探索方面，南京航空航天大学 2013 年的硕士学位论文走在了前面。他们在 320×240 分辨率的低性能嵌入式平台上，用 HOG 加 SVM 分类器^[5]做出了车载固定目标识别，单发命中精度达到 0.8 mil。这说明传统机器学习在资源受限场景下是可行的。不过，这项工作没能有效解决“边走边打”（也就是动基座打静止目标）过程中平台振动带来的零偏问题——零偏往往在 100 ms 量级，足够把瞄准精度毁掉，这个问题后来成了后续研究的一个重点。

到了深度学习时代，齐鲁工业大学 2021 年的学位论文往前迈了一大步。他们把 Faster R-CNN^[2] 的骨干网络换成轻量化的 MobileNet，在 STM32 加 OpenMV 的架构上跑出了 20 FPS 的实时检测，mAP 达到了 72%，比 HOG+SVM 明显好。但一个突出的工程短板也暴露出来：遮挡场景或者强光突变下，目标丢失率仍然有 6% 8%，系统的鲁棒性还是不够。

电子科技大学 2022 年的一项嵌入式视觉研究，把这个瓶颈进一步量化了。他们仔细分析了性能数据：就算用轻量化的 MobileNet CNN，在 640×480 的分辨率下，推理过程仍然会吃掉嵌入式 GPU 80% 的算力。这种高负载不光导致瞄准延迟超过 100 ms（实时性要求可是 10 ms 量级），还会带来明显的功耗上升和热量积累，威胁嵌入式平台的稳定性。

算法层面，国内在 IMU 数据处理上也有进展。好几个团队研究了 IMU 零偏校准和温漂补偿，但多数还停在离线标定阶段，实时的自适应标定和在线零偏补偿很少见。至于“像素误差加角速度前馈”这种紧耦合双闭环控制，在国内还处于萌芽期，缺乏系统的理论分析和工程验证。

总的来说，国内轻量化方案在振动抑制、实时性和鲁棒性上，跟国外大概有 1 到 2 年的工程化差距。原因主要有三个：一是国内团队在算法创新上积累不够，习惯跟着国外最新论文走，工程适配性没跟上；二是轻量化 IMU 和高性能嵌入式芯片在国内不太好拿；三是“纯视觉加 IMU”紧耦合的完整系统设计和验证，缺少学科交叉的系统研究。

1.3.3 存在的主要问题

综合国内外研究现状，当前存在的主要问题包括：

1. 国外方案在成本、体积和功耗方面存在极端不平衡，不适用于轻量化应用；
2. 国内轻量化方案在振动抑制、实时性与鲁棒性上与国外存在 1~2 年的工程化差距；
3. “像素-角速度”一体化设计尚处萌芽阶段，“纯视觉 +IMU”紧耦合的轻量化方案缺乏系统研究；
4. 移动平台动态运动对瞄准精度的影响补偿不足，需要建立双闭环控制架构。

1.3.4 本文的设计思路

综合以上分析，本工作确定了以下几条设计原则：

1. **架构选择**：走“纯视觉 +IMU”紧耦合的路线，不碰激光雷达这种昂贵的传感器，在“千元级成本、5 W 功耗”的框框里做出可接受的精度；
2. **特征算法**：用 HOG、ORB 这类传统特征和模板匹配，不依赖 GPU，在嵌入式平台上就能跑，保证帧率超过 25 FPS；

3. **控制架构**：搭一套“像素误差外环加角速度前馈内环”的双闭环，靠 IMU 数据提前预估图像偏移，把平台机动带来的误差降下来；

4. **时间同步**：硬件层面让图像和 IMU 数据的时间戳对齐，给紧耦合打好底。

这样设计，既保证了工程上的可复现性，也为将来技术升级留了余地。

1.4 发展趋势

国内移动平台瞄准系统正在从“多传感器融合”向“视觉-IMU”紧耦合的轻量化方向转变。硬件上，国产微控制器的主频已经突破了 480 MHz，内置 FPU 和 DSP 指令集，给“视觉跟踪加 IMU 前馈”同频运行提供了算力保障。算法上，HOG、ORB 这类传统特征不要 GPU 就能跑出高帧率，而且跟 IMU 采样频率接近，时间戳对齐也方便。控制上，“像素误差外环、角速度前馈内环”的双闭环已经比较成熟，能把平台机动引起的瞄准线误差压缩掉 30% 以上。

可以说，“纯视觉加 IMU”紧耦合已经是低成本动基座瞄准的一个明确趋势，也给本科毕业设计提供了一个能落地、能复现的技术窗口。

1.5 主要内容和论文组织

本文围绕移动平台固定目标识别与瞄准问题，设计并实现了一套低成本系统。全文 6 章，内容安排如下：

第 1 章（引言）交代课题的研究背景和应用意义，梳理国内外研究现状、存在的问题、设计思路和发展趋势，说明为什么做这个课题、能不能做。

第 2 章（系统总体设计）给出系统整体方案，明确“Linux 视觉端 + 云台主控 + 底盘主控”的三节点分布式架构，说明各节点之间的通信协议和数据流、系统工作原理和框图，同时列出功能需求和性能指标约束。

第 3 章（系统硬件设计）逐一讲解硬件各模块的详细设计，包括 STM32H7 主控核心电路、HiPNUC 六轴 IMU 采集电路、USB 摄像头接入方式、MS4010 双轴云台驱动电路和底盘传感器接口，最后给出整机连接关系图。

第 4 章（系统软件设计）详细描述软件的分层架构和实现细节——IMU 角速度采集与处理、视觉目标检测与跟踪、像素误差外环和 IMU 前馈内环双闭环控制算法、底盘差速驱动与八路灰度循迹控制，以及节点间的通信协议实现。

第 5 章（系统调试与结果分析）分模块和整机两个层次进行调试，包括图像识别与跟踪效果测试、IMU 采样稳定性与补偿性能验证、云台闭环稳态误差测试和底盘循迹精度分析，给出实测结果和相应分析。

第 6 章（总结与展望）归纳主要工作和创新点，分析系统当前存在的不足，并讨论后续可以深化的方向和技术升级路径。

第2章 系统总体设计

本章从整体上交代系统的设计方案，重点讲清楚工作原理、各模块之间的关系和性能上的限制条件，为后面器件选型、硬件实现和软件设计打好统一的基础。

2.1 系统设计工作原理及原理框图

2.1.1 基本工作原理

系统以“视觉感知+惯性测量+双闭环控制”为核心链路：摄像头完成目标图像采集与误差提取，IMU提供高频角速度信息，控制器根据像素偏差与角速度前馈生成云台控制量，实现对固定目标的持续指向与误差抑制。

在执行过程中，系统按照“采集-估计-控制-反馈”循环运行。外环以像素误差为主，内环以角速度补偿为主，从而兼顾稳态精度与动态响应。从计算节点角度看，系统由三个协同工作的处理单元构成：

(1) **Linux 视觉节点**：以通用计算平台运行 OpenCV 视觉管线，完成目标检测、位姿解算与跟踪状态管理，结果通过串口协议下发给云台主控；

(2) **云台主控节点 (STM32H7)**：以 480 MHz 主频实时处理 IMU 数据、执行双轴伺服控制和瞄准解算，并通过 RS485 总线驱动 MS4010 双轴电机；

(3) **底盘主控节点 (独立单片机)**：独立执行八路灰度传感器循迹、编码器 PI 速度闭环与圈次任务，与云台节点通过接口协同而非控制耦合，互不干扰。

该三节点架构将视觉计算、姿态控制和底盘机动分散到各自最适合的平台，降低单节点负担的同时提升系统可扩展性。

2.1.2 系统原理框图与模块说明

系统总体由移动底盘、云台执行机构、视觉采集模块、IMU 模块、主控与通信模块、上位机监控模块构成。各模块间通过标准接口进行数据交互，形成可扩展、可维护的分层结构。系统总体框图如图2.1所示。

如图所示，系统分为三个功能子系统：瞄准模块（STM32H7 云台控制 MCU + 陀螺仪 + 云台电机）、视觉模块（摄像头 + 树莓派/Linux 视觉节点）以及自动循迹小车（小车控制 MCU + 八路灰度传感器 + 小车电机）。各子系统通过串口/RS485 互联，以 12 V 电源总线统一供电。

其中，主控侧负责实时控制与状态发布；上位机侧负责参数配置、可视化监测与实验数据记录。该分层架构便于后续算法替换与功能迭代。

在节点间通信层面，各链路数据类型与协议各异：Linux 视觉节点以固定帧格式 @px,py,flag,# 通过串口向云台主控输出目标像素坐标与置信标志；云台主控通过 RS485 总线与 MS4010 电机通信，完成角度指令打包、响应解析与超时检测；底盘主控以独立控制器实现内部闭环，与上层通过预定义接口完成模式同步。这种“链路分离”设计使各子系统在对方降级时仍能维持最小功能闭环，提升了整体鲁棒性。

2.2 系统设计要求

2.2.1 功能性要求

系统应具备目标识别与连续跟踪、云台姿态控制、实时状态回传、参数在线整定等基础功能。核心任务是在移动平台运动扰动下保持瞄准线稳定。

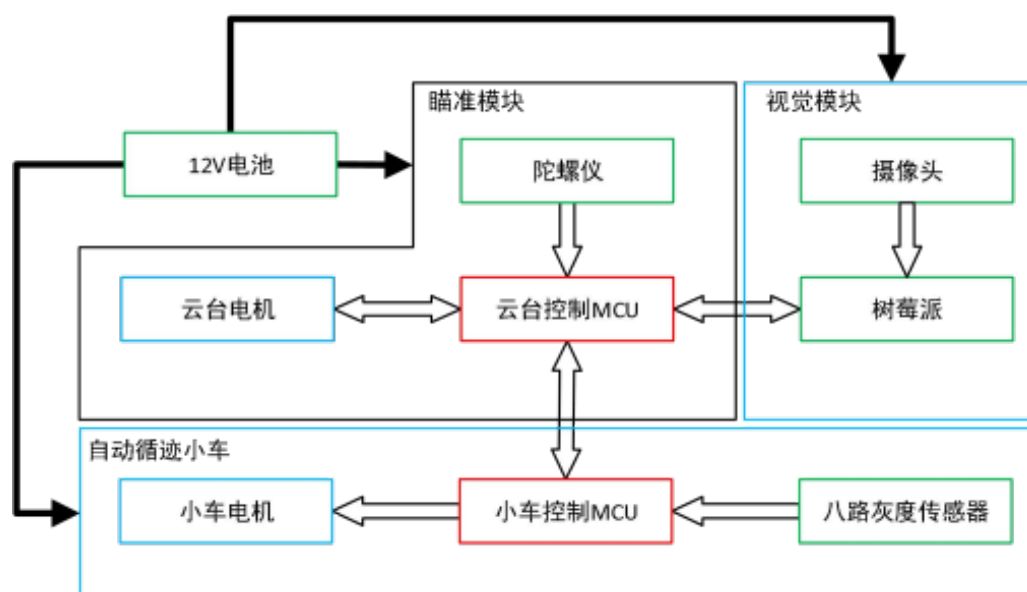


图 2.1 系统总体框图

2.2.2 性能指标要求

结合开题目标，系统设计以低成本平台为约束，重点满足以下定量指标，如表2.1所示。

指标类别	指标项	设计目标	说明
视觉实时性	视觉帧率	$\geq 25 \text{ FPS}$	室内标准光照场景
惯性采样	IMU 采样频率	$\geq 100 \text{ Hz}$	角速度前馈需求
瞄准精度	稳态指向误差	$\leq 0.8 \text{ mrad}$	静态目标双闭环收敛后
成本约束	整机器件成本	千元级 ($\leq \text{¥}1000$)	不含通用计算平台
功耗约束	系统功耗	$\leq 5 \text{ W}$	嵌入式部分
重量约束	嵌入式系统重量	$\leq 500 \text{ g}$	含云台机构

表 2.1 系统性能指标要求

其中， 0.8 mrad 的稳态误差指标来源于对典型室内应用场景的分析； 25 FPS 视觉帧率保证像素闭环具备足够带宽裕量； 100 Hz IMU 采样频率则确保前馈补偿对平台机动具有及时响应能力。

2.2.3 约束条件与设计边界

系统设计受限于本科毕设条件与嵌入式算力，遵循“低成本、低功耗、可复现”的工程边界，不依赖高功耗 GPU 与昂贵传感器。在此约束下，优先选择成熟、易调试、易复现的实现路线。

2.3 关键器件选型与接口设计

为落实总体设计目标，本节在“可获取性、接口兼容性、实时性、工程可复现性”原则下完成关键器件选型与接口关系定义，使系统方案从架构层进一步落到可实现的工程层。

2.3.1 主控制器件

云台控制系统需要处理来自 IMU 和视觉模块的多源数据，并在实时控制周期内完成角速度前馈补偿与伺服控制的计算。因此主控制器件必须具备较高的主频、充足的存储资源和完善的外设接口，同时需要支持定时控制、中断管理和通信协议扩展，并具有健全的开发生态以便快速开发。

结合这些需求，本课题选用 STM32H7 系列微控制器作为云台主控平台。STM32 系列中 STM32H743XIH6 的 CPU 主频高达 480 MHz，内置单精度与双精度浮点运算单元 (FPU) 及 DSP 指令集，能够承担 IMU 数据处理、控制律计算与通信管理等高实时性任务。同时 STM32H7 生态成熟，支持移植 RT-Thread 实时操作系统，便于快速调用各类驱动资源进行高效开发。相较于其他微控制器方案（如 MSPM0G3507 主频仅 80 MHz），STM32H7 在计算能力和浮点性能上具有显著优势，更能满足云台双轴伺服、IMU 前馈补偿与视觉数据融合需要同频运行的实时控制需求。

2.3.2 传感器与视觉模块

系统采用六轴惯性测量单元 (IMU) 用于姿态感知与角速度前馈，其输出数据为动态补偿提供了高频的输入信息，能够在平台运动状态下有效提升瞄准稳定性。经过零偏校准与滤波处理后，惯性传感器的测量精度可以进一步提高。选取 IMU 时需要确保角速度测量稳定、采样频率可灵活配置、通信接口可靠且抗干扰能力强，同时与主控平台在通信协议和电气特性上兼容。

在视觉模块方面，课题采用 USB 摄像头作为图像输入源。与专用视觉芯片模块（如 MaixCAM 等）相比，USB 摄像头的部署成本低、驱动生态完善，便于与上位机图像处理流程集成。更重要的是，USB 摄像头配合 Linux 平台能够直接运行 OpenCV 传统视觉算法，在本课题以 HSV 色彩分割和 PnP 位姿解算为核心的目标识别链中，帧率表现稳定且一致性好。此外，摄像头体积小、重量轻，易于架设在云台机构上随动，符合轻量化的系统设计目标。

2.3.3 执行机构与接口设计

云台的执行机构需要具备快速响应、精度可调、驱动稳定和维护简便等特点，并须与主控的输出信号相互匹配。在电机方案选择中，本课题对比了两种技术路线：其一为张大头自闭环步进电机云台，具有精度高、通讯简单的优点，但两轴步进电机组合后机械体积偏大，不利于整机的轻量化设计；其二为采用 RS485 接口的小型无刷云台机构，在满足控制精度要求的同时具备更为紧凑的体积和更轻的重量，与 STM32H7 的输出信号具有良好的兼容性。综合结构、性能等多方面因素，选择方案二作为执行机构。通过合理的驱动电路设计与闭环控制策略，可实现方位与俯仰通道的协调控制。

系统的信号连接采用 RS485 总线串口通讯方式。STM32H743 通过 UART 外设与 RS485 收发器相连，按照 MS4010 协议向偏航轴 (ID1) 和俯仰轴 (ID2) 的电机发送速度或位置指令帧。相比于 PWM 驱动方案，RS485 总线具有更强的抗干扰能力和多节点扩展潜力，通信速率可达 1 Mbit/s，足以满足 500 Hz 的控制周期实时要求。清晰的接口定义规范有利于后续系统联调复杂度的降低。

同时，本系统充分利用 STM32H7 的丰富外设资源：串口用于与 RS485 模块和通信设备通讯，定时器负责周期性的控制任务调度，PWM 模块驱动执行器，而 I2C 或 SPI 接口用于传感器数据采集。IMU 与主控通过标准数字接口完成数据交互，摄像头则与上位机侧构成独立的图像采集链路。两路数据流在控制层按时间顺序进行融合处理，保证控制计算的时间一致性和数据同步性。

2.4 本章小结

本章围绕总体设计主线，完成了系统工作机理、三节点分布式架构设计、定量性能指标约束以及关键器件选型与接口关系定义，形成了从”方案定义”到”工程落地”的完整闭环，为下一章硬件详细设计与实现提供了直接依据。

第3章 系统硬件设计

本章在总体架构和器件选型基础上，给出各硬件子模块的电路设计思路与连接关系，明确系统从器件到整机的实现路径。

3.1 单片机控制核心电路

主控电路包含最小系统、电源管理、时钟与下载调试接口。设计中重点保证供电稳定、复位可靠与信号完整，以提高系统连续运行能力。

瞄准模块以 STM32H743 核心板为中心，外围扩展 RS485 电机总线接口、HiPNUC IMU 串口接口和视觉串口接口。电路原理图如图3.1所示，PCB 布局如图3.2所示。

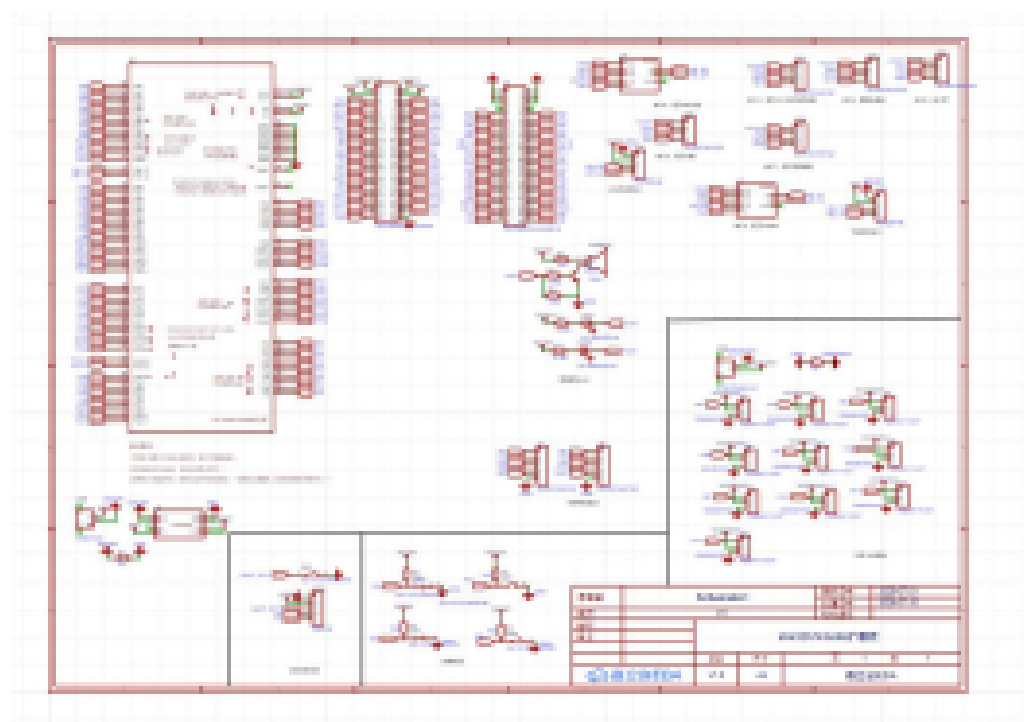


图 3.1 瞄准模块电路原理图

原理图中，STM32H7 通过若干 GPIO 扩展了电机 RS485 收发器、IMU UART 接口及指示 LED 等外围器件；PCB 布局充分考虑了高频信号与电源回路的分离，减少干扰。图3.3为所用 STM32H743 核心板实物图，芯片封装为 LQFP144，板载 Winbond W9825G6KH-6I 外扩 SDRAM，具备充足的运行内存供 RT-Thread 操作系统及多线程控制任务使用。

3.2 IMU 数据采集电路

3.2.1 IMU 内部结构与工作模式

系统选用 HiPNUC 六轴 IMU 传感器，内置三轴加速度计与三轴陀螺仪，通过片内算法完成姿态解算，可直接输出角速度、姿态角与加速度等物理量。该传感器采用 UART 接口与主控通信，默认波特率 921600，输出频率可按需配置，典型工作模式下输出频率

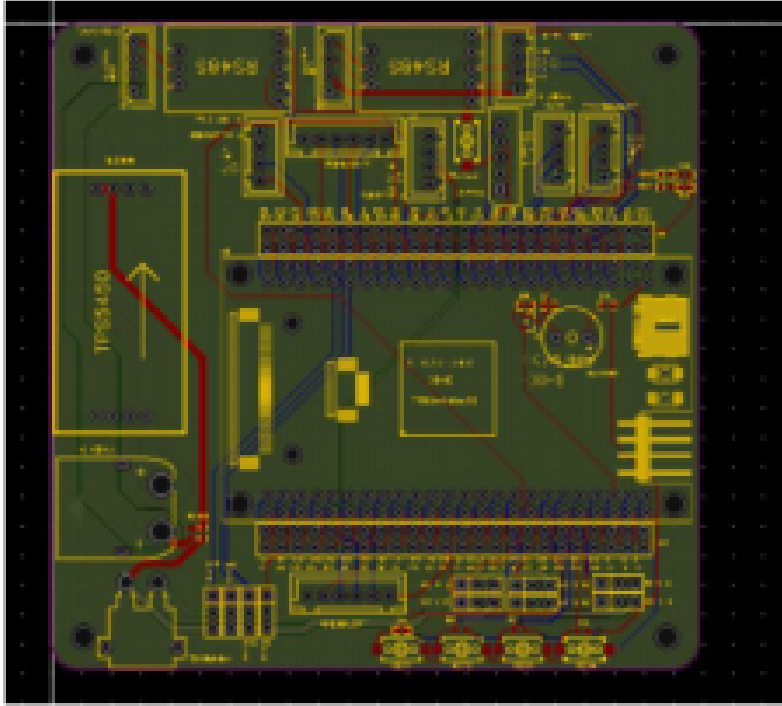


图 3.2 瞄准模块电路 PCB 布局

设置为 100 Hz 以匹配控制周期要求。

通过合理配置量程与滤波参数，可在兼顾动态响应与噪声抑制的同时，为后续前馈补偿提供高质量数据。

3.2.2 IMU 时序与采样配置

控制周期设置为 10 ms（对应采样频率 100 Hz），采用定时触发与 DMA 中断读取机制保证数据时序稳定。主控通过 `hipnuc_dec` 模块对 HiPNUC 协议进行字节流解码：先执行帧同步与 CRC 校验，再按数据包类型提取角速度 ω 、姿态角 θ 与加速度数据。解析成功后数据被发布到控制路径，供 IMU 前馈补偿计算使用。

对于角速度小幅波动，工程采用死区处理策略，避免将传感噪声直接放大到执行端，提升闭环控制品质。

3.2.3 IMU 接口连接与抗干扰设计

在接口连线中考虑去耦、电源滤波与布线隔离，降低电机驱动等强干扰源对惯性数据的影响，提升测量可靠性。

3.3 图像采集与通信电路

3.3.1 USB 摄像头接入原理

摄像头通过 USB 链路与上位机连接，形成图像采集与处理通道。该方案可降低主控侧图像处理压力，便于快速验证视觉算法。

为消除镜头几何畸变对 PnP 位姿解算的影响，需对摄像头进行标定。本文采用张正友棋盘格柔性标定方法^[6]，经棋盘格标定实验，所用摄像头的内参矩阵如式(3.1)所示：

$$\mathbf{K} = \begin{bmatrix} 435.37 & 0 & 318.68 \\ 0 & 433.86 & 226.72 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.1)$$

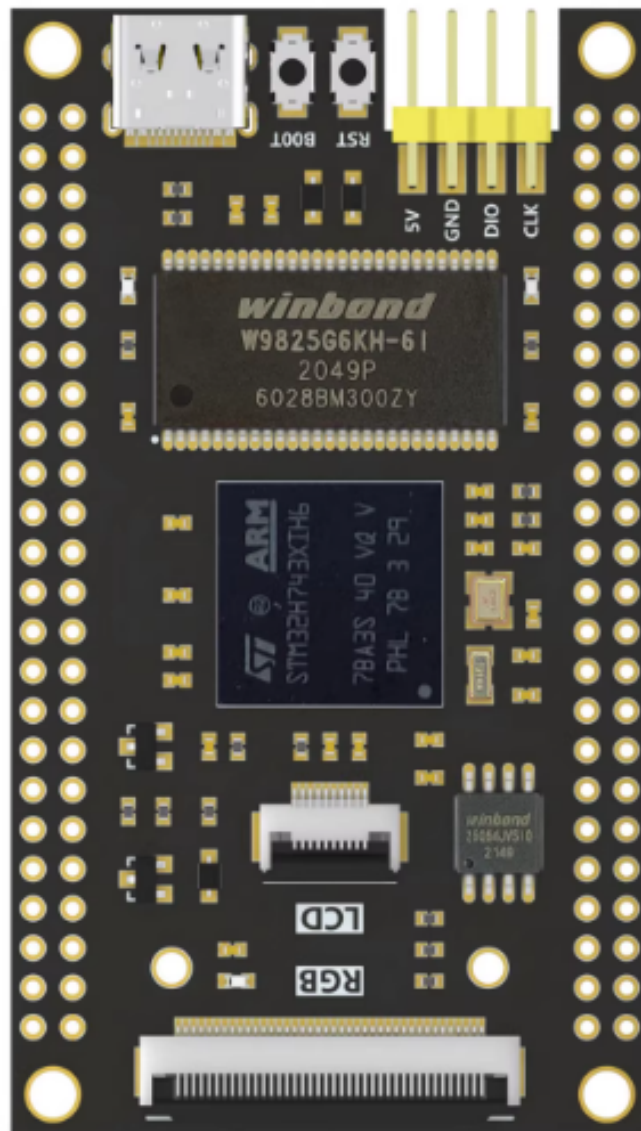


图 3.3 STM32H743 核心板实物图

其中, $f_x=435.37\text{ px}$ 、 $f_y=433.86\text{ px}$ 为焦距, $(c_x, c_y)=(318.68, 226.72)\text{ px}$ 为主点坐标。标定还同步获得径向畸变系数, 标定后重投影误差可降至次像素级别, 满足后续 A4 纸角点定位精度需求。

3.3.2 图像数据链路设计

图像链路设计关注带宽、延迟与稳定性, 确保视频流在实验帧率下可持续传输, 为实时跟踪与控制反馈提供数据支持。

3.4 云台驱动与电机控制电路

3.4.1 电机驱动原理

云台采用 MS4010 双通道电机驱动结构。偏航轴 (Yaw) 对应电机 ID 1, 俯仰轴 (Pitch) 对应电机 ID 2, 两轴通过 RS485 总线与主控通信, 可实现位置模式与速度模式的在线切换, 满足指向和跟踪两类控制需求。

系统上电后, 主控扫描 RS485 总线发现 ID 1 与 ID 2 两台电机, 完成云台初始化参数绑定、角度限位设置与回零动作, 随后进入待命状态等待上层控制指令。MS4010 v3 电机实物如图3.4所示, 机身侧面的 DIP 拨码开关用于设定 RS485 设备地址 (即电机 ID), 四位拨码对应二进制编码; 底部印有 RS485 差分信号接线标识及电源正负极标注, 安装时需严格核对极性以防驱动器损坏。



图 3.4 MS4010 v3 云台步进电机实物图

3.4.2 RS485 总线接口与电源设计

云台电机的控制信号经由 UART 外设驱动 RS485 收发器芯片后发出, MS4010 电机以差分信号接收指令, 并将编码器状态通过同一总线回报。设计中对供电冗余、地线回流与过流保护进行约束, 总线终端匹配电阻置于链路末端, 消除长线反射, 提升整机稳定性。

系统采用 12 V 锂电池统一供电, 经 TPS5400 DC-DC 降压模块分别产生 5 V 与 3.3 V 轨为各子模块供电。电源电路原理图与 PCB 如图3.5和图3.6所示。

电源设计采用双路 TPS5400 降压芯片, 保证各节点在满载条件下的供电稳定性; 接线端子处增加滤波电容, 抑制电机启停产生的电源纹波对信号链路的干扰。

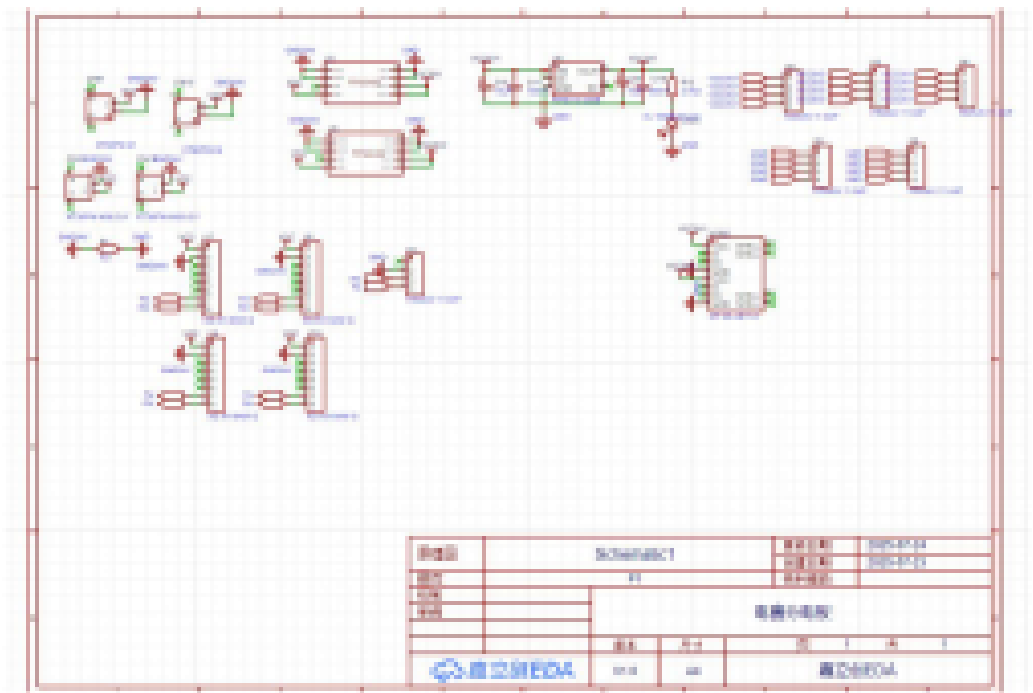


图 3.5 电源电路原理图

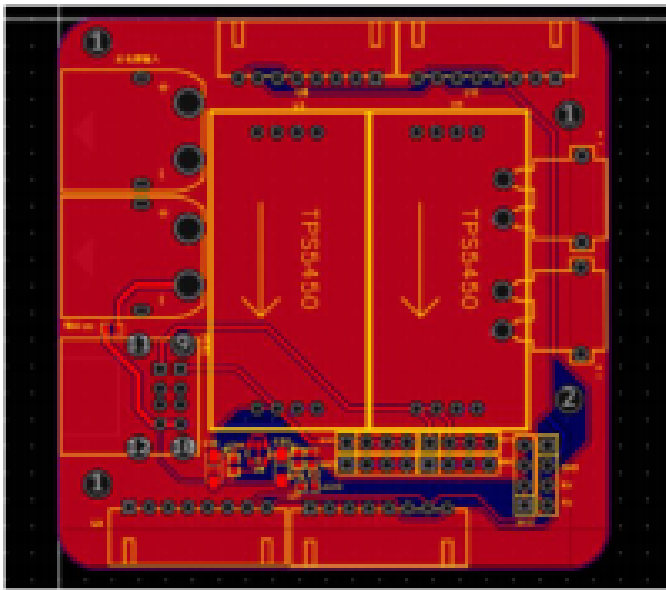


图 3.6 电源电路 PCB 布局

3.5 自动循迹底盘控制电路

底盘子系统以 MSPM0G3507（Arm Cortex-M0+，80 MHz）为主控，连接八路模拟量灰度传感器与 MG513X 减速电机编码器。相比数字量灰度传感器，模拟量传感器可输出连续电压信号，内置自动校准功能，无需每次重启重新标定阈值，更有利于差速 Cloud PID 控制的丝滑运行。底盘控制电路原理图与 PCB 布局如图3.7和图3.8所示。

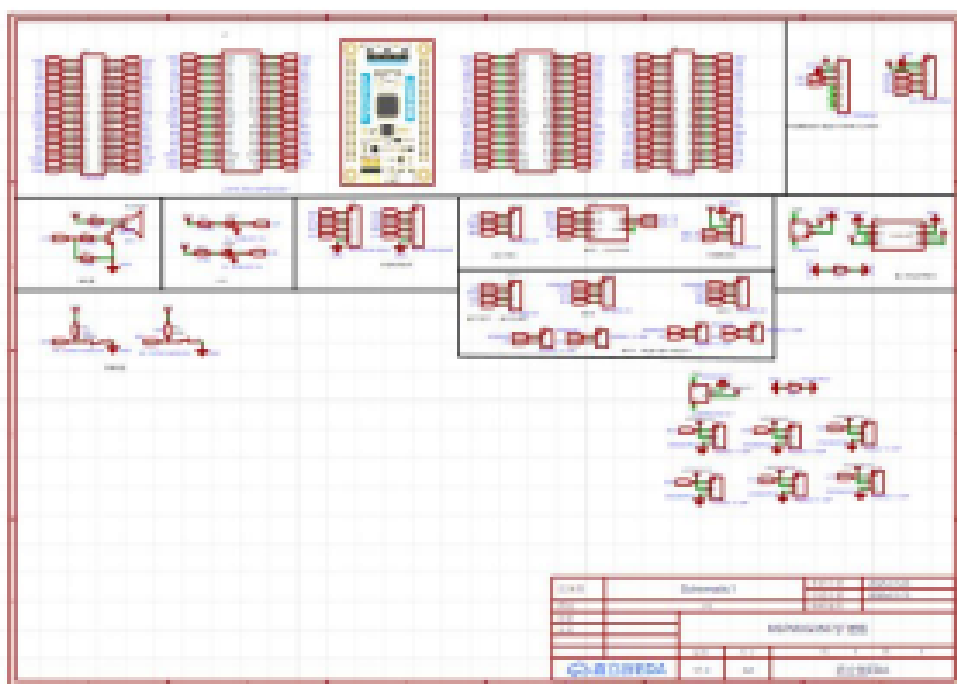


图 3.7 自动循迹底盘电路原理图

底盘 PCB 集成了 MSPM0G3507 核心、电机驱动 H 桥电路、灰度传感器 ADC 接口与编码器采样电路；板载 TPS5400 为主控和传感器分别提供稳压；布局时将大电流电机驱动区与信号采集区隔开，降低电磁干扰。图3.9为 WHEELTEC C07A 底盘核心板实物图，核心芯片为 MSPM0G3507，板上集成两颗用户按键可触发运行模式切换，电机驱动与灰度传感器接口通过排针引出，便于扩展与更换。

3.6 系统硬件联调与整机连接图

在整机联调阶段，对模块连接、接口映射和供电路径进行一致性检查，形成完整硬件连接关系图，并验证系统基本功能可运行。

系统三个计算节点间的物理连接关系如下：Linux 视觉节点（上位机）通过 USB 接摄像头完成图像采集，同时通过 UART 串口与 STM32H7 云台主控相连，发送目标像素数据；STM32H7 主控通过 UART 接收 HiPNUC IMU 数据，通过 RS485 总线驱动 MS4010 双轴电机（偏航轴 ID1、俯仰轴 ID2）；底盘主控节点独立连接八路灰度传感器阵列与左右编码器，承担差速机动执行。整机供电采用统一电源总线，各节点自行管理本地去耦与保护，避免强电设备对信号回路的干扰。

3.7 本章小结

本章完成了主控、传感、视觉、执行与保护模块的硬件设计说明，明确了整机连接和联调重点，为软件开发与系统测试提供硬件基础。

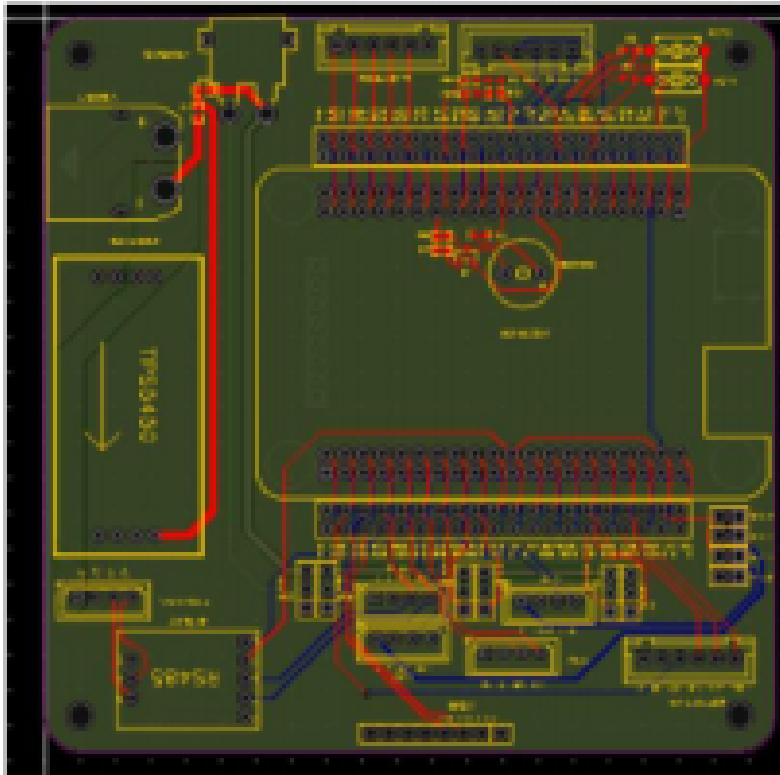


图 3.8 自动循迹底盘电路 PCB 布局

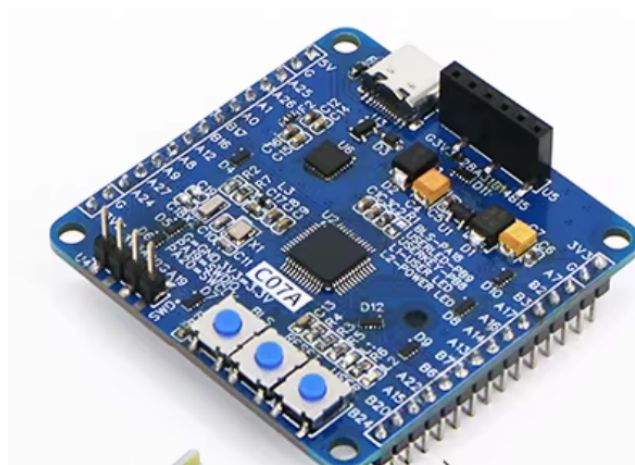


图 3.9 WHEELTEC C07A 底盘核心板（MSPM0G3507）实物图

第4章 系统软件设计

本章基于“Linux 视觉端 + 云台主控 + 底盘主控”的分布式实现方案，围绕“感知-决策-执行”闭环，说明系统软件架构、跨节点协同流程、控制算法、通信协议与异常处理机制。相较于单机式流程描述，本章强调多控制器协同下的可复现实装路径，即各功能模块在 Linux 与 MCU 异构平台上的职责边界与交互方式。

4.1 软件总体架构与流程

4.1.1 程序分层与任务划分

本课题软件采用分布式分层架构，系统由三个相互独立但协同工作的计算节点组成：

- (1) **Linux 视觉节点**：负责目标检测/识别、目标状态管理与视觉数据封装输出；
- (2) **云台主控节点 (STM32)**：负责 IMU 融合、双轴伺服控制、瞄准解算与云台执行闭环；
- (3) **底盘主控节点 (独立单片机)**：负责路径循迹、速度闭环、转向计数与圈次任务执行。

在此基础上，按功能可划分为以下五层：

- (1) **设备驱动层**：完成 RS485 电机通信、IMU 数据解码与串口接收，核心模块包括 `drv_ms4010`、`hipnuc_dec` 和 `usart_dataparsing`。
- (2) **运动控制层**：由 `laser_gimbal` 实现双轴角度控制、速度控制、坐标指向、回零与限位管理。
- (3) **视觉追踪层**：Linux 侧完成目标检测与状态跟踪，云台侧 `laser_gimbal_vision_tracker` 完成像素误差闭环、目标状态机与自动搜索逻辑。
- (4) **系统集成层**：在 `main.c`、`laser_gimbal_test.c` 和 `laser_gimbal_vision_test.c` 中完成云台侧模块初始化与运行控制，同时通过串口/总线协议与 Linux 视觉端、底盘主控端进行数据协同。
- (5) **调试交互层**：通过 MSH 命令导出接口实现在线调参与功能验证，支撑不同场景下的快速迭代。

这种“节点解耦 + 功能分层”的方式，将视觉计算负载、姿态控制任务和底盘实时控制任务分散到各自最适合的平台上，既降低单节点计算压力，也提升了系统可扩展性与工程可维护性。

4.1.2 软件主流程图

分布式系统主流程遵循“节点自检-链路建立-闭环协同”的启动顺序：

- (1) **底盘节点启动**：完成电机、编码器、灰度传感器初始化，进入待命模式；
- (2) **云台节点启动**：初始化 GPIO 与基础外设，调用 `laser_gimbal_system_init` 完成电机管理器建立、电机扫描、云台参数绑定、电机使能与回零；
- (3) **视觉节点启动**：Linux 侧加载视觉模型与目标管理模块，建立与云台主控的数据通道并开始输出目标信息；
- (4) **闭环协同运行**：云台节点持续接收视觉目标输入与 IMU 状态量，计算双轴控制量并执行瞄准；底盘节点独立执行循迹与速度闭环，同时按任务状态与上层指令完成机动。系统总体程序流程图如图4.1所示，左侧为视觉与云台控制链路，右侧为底盘循迹控制链路，两条链路并行运行、通过接口协同。

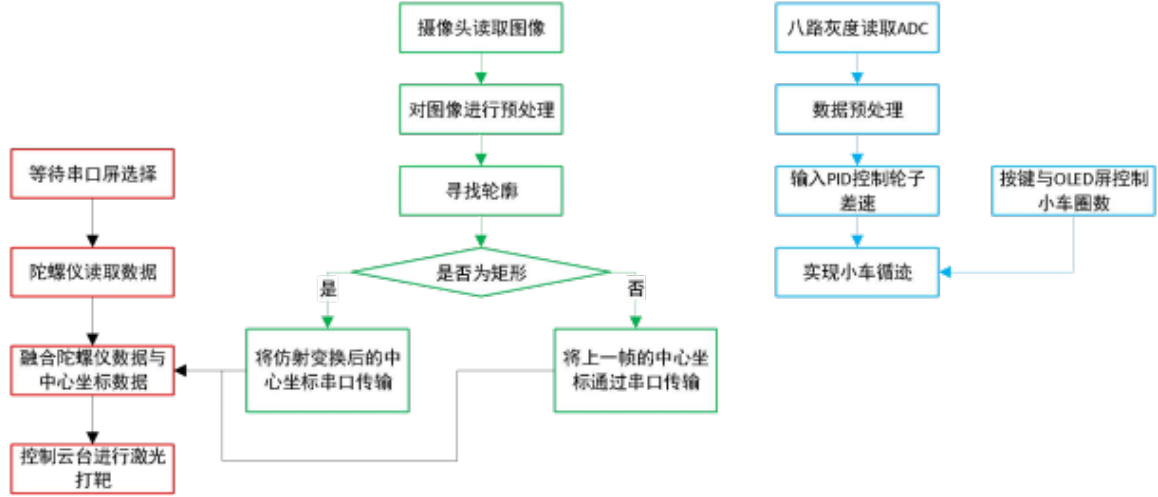


图 4.1 系统总体程序流程图

该流程体现了“分布式职责清晰、局部闭环自治、跨节点协议协同”的工程原则：即使某一节点短时降级，其他节点仍可维持各自最小功能闭环，从而提升系统鲁棒性并降低联调复杂度。

4.2 IMU 数据处理程序

4.2.1 数据采集与滤波

系统通过 `hipnuc_dec` 模块对 HiPNUC 协议进行字节流解码，先执行帧同步与 CRC 校验，再按数据包类型提取角速度、姿态角与加速度。数据解析成功后，角速度 ω 与姿态角 θ 被发布到控制路径供补偿计算使用。

在实现层面，工程采用“协议解码 + 阈值抑噪”策略：对小幅波动进行死区处理，避免将传感噪声直接放大到执行端。

4.2.2 姿态解算与角速度发布

系统在追踪线程中读取 IMU 实时状态，并构建前馈补偿项。以偏航轴为例，控制输出如式(4.1)所示，该结构参考^[7,8]中自抗扰控制思想，在传统 PID 外环基础上叠加角速度与加速度前馈项：

$$u_{yaw} = u_{pid} + k_v \cdot \omega_{yaw} + k_a \cdot \dot{\omega}_{yaw} + k_x \cdot a_x \quad (4.1)$$

其中， u_{pid} 为视觉误差闭环输出， ω_{yaw} 为角速度项， $\dot{\omega}_{yaw}$ 为角加速度近似项， a_x 为加速度补偿项。该结构能够在平台突加速或抖动时提前修正执行量，降低纯视觉闭环的相位滞后影响^[9,10]。

代码4.1展示了追踪控制线程的核心逻辑：每 2 ms 一次（约 500 Hz）读取 IMU 数据计算前馈量，与视觉 APID 输出叠加后写入步进电机驱动器。

```

1 static void vision_tracker_thread_entry (void *parameter)
2 {
3     vision_tracker_t *tracker = (vision_tracker_t *)parameter;
4     float yaw_out = 0, pitch_out = 0;
5
6     while (tracker->thread_running) {
7         yaw_out = 0; pitch_out = 0;
8     }

```

```

9      if ( tracker->enabled) {
10         if (rt_mutex_take( tracker->tracker_mutex, 10) == RT_EOK) {
11             /* 读取IMU角速度/加速度前馈量 */
12             extern hipnuc_raw_t hipnuc_raw;
13             float kv = kp_vel_yaw * hipnuc_raw.hi91.gyr[2];
14             float ka = kp_angle_yaw * hipnuc_raw.hi91.acc[0];
15
16             /* 视觉APID外环输出 */
17             vision_tracker_process_target ( tracker ,
18                                             &yaw_out, &pitch_out);
19
20             /* 叠加IMU前馈，补偿平台抖动 */
21             yaw_out = yaw_out + kv + ka;
22
23             vision_tracker_apply_control ( tracker , yaw_out, pitch_out);
24             rt_mutex_release ( tracker->tracker_mutex);
25         }
26     }
27     rt_thread_mdelay(2); /* 控制周期~500 Hz */
28 }
29 }

```

代码 4.1: 视觉追踪线程与 IMU 前馈补偿 (laser_gimbal_vision_tracker.c)

4.3 视觉识别与跟踪程序

4.3.1 视觉线程架构与数据流

根据参考目录下 edc_opencv/EDC_Openencv.py 的实现，视觉端采用“多线程 + 双队列”结构：

(1) 读取线程 read_thread_func: 持续从摄像头读取帧，并通过 frame_queue(maxsize=1) 仅保留最新一帧，降低处理时延累积；

(2) 处理线程 process_thread_func: 完成颜色阈值分割、轮廓筛选、A4 目标判别、PnP 位姿解算与目标状态管理；

(3) 发送线程 send_thread_func: 从 data_queue 取出结果并按串口协议发送给云台主控。

该结构将“采集实时性”“算法计算量”和“通信阻塞风险”解耦，使视觉链路在高帧率下仍具备稳定吞吐。其核心数据路径可概括为：

Camera Frame → HSV Segmentation → Contour Filter → PnP Pose → Serial Packet

读取线程核心代码如代码4.2所示，通过 maxsize=1 的有界队列确保处理端始终获取到最新帧，避免帧积压导致延迟累积。

```

1 def read_thread_func(cap):
2     """线程1: 读取视频数据，仅保留最新一帧"""
3     while running_event.is_set() and cap.isOpened():
4         ret, frame = cap.read()
5         if not ret:
6             running_event.clear()
7             break
8     try:

```

```

9         # 清空队列，确保只保留最新的一帧
10        while not frame_queue.empty():
11            frame_queue.get_nowait()
12            frame_queue.put_nowait(frame)
13    except queue.Full:
14        pass # 并发竞争时忽略

```

代码 4.2: 视觉读取线程 (EDC_Opencv.py)

发送线程负责将处理结果格式化为 @x,y,flag,# 协议帧并通过串口发送，核心代码如代码4.3所示。

```

1  def send_thread_func(ser):
2      """线程3: 从队列取结果并按协议发送"""
3      while running_event.is_set():
4          try:
5              data = data_queue.get(timeout=1)
6              # 固定帧格式: @x,y,flag,#
7              msg = f"@{data['x']:.3 f},{data['y']:.3 f},{data['flag']:.1 f},#"
8              if ser and ser.is_open:
9                  ser.write(msg.encode('utf-8'))
10         except queue.Empty:
11             continue

```

代码 4.3: 视觉发送线程与串口协议帧 (EDC_Opencv.py)

4.3.2 目标检测与几何筛选流程

处理线程首先将图像统一到 640×480 并进行高斯滤波，随后在 HSV 空间执行 inRange 二值化和开运算去噪。阈值由 Trackbar 在线调节并可写回 trackbar_values.txt，便于现场快速整定。

轮廓处理遵循“先粗筛、后精筛”策略：

(1) 多轮廓合并：merge_close_contours 按面积差与质心距离合并近邻轮廓，抑制边缘断裂造成的重复候选；

(2) 几何约束：要求候选轮廓近似四边形，面积位于设定区间且宽高比满足约束；

(3) 尺寸一致性校验：find_a4_paper_contours 利用 solvePnP 和重投影尺寸估计，判断候选是否与 A4 目标物理尺寸一致。

仅当上述条件同时满足时，目标才进入后续位姿解算与下发链路。该“形状 + 尺度”双重判据可显著降低误检。几何筛选与 PnP 尺寸核查的关键代码如代码4.4所示。函数先通过 solvePnP 解算位姿，再利用重投影尺寸估计与 A4 物理尺寸比对，拒绝误匹配的候选轮廓。

```

1  def find_a4_paper_contours(contour, size_tolerance_mm=50):
2      object_points = np.array([
3          [-A4_WIDTH_MM/2, -A4_HEIGHT_MM/2, 0],
4          [A4_WIDTH_MM/2, -A4_HEIGHT_MM/2, 0],
5          [A4_WIDTH_MM/2, A4_HEIGHT_MM/2, 0],
6          [-A4_WIDTH_MM/2, A4_HEIGHT_MM/2, 0]
7      ], dtype="float32")
8      ordered_pts = order_points(contour)
9      (success, rvec, tvec) = cv2.solvePnP(object_points, ordered_pts,
10                                          CAMERA_MATRIX, DIST_COEFFS)
11      if success:

```

```

12 distance_z_mm = tvec[2][0]
13 if distance_z_mm < 100 or distance_z_mm > 2000:
14     return False
15 reprojected, _ = cv2.projectPoints(object_points, rvec, tvec,
16                                   CAMERA_MATRIX, DIST_COEFFS)
17 reprojected = reprojected.reshape(-1, 2)
18 w_px = np.linalg.norm(reprojected[1] - reprojected[0])
19 h_px = np.linalg.norm(reprojected[3] - reprojected[0])
20 est_w = (w_px * distance_z_mm) / CAMERA_MATRIX[0, 0]
21 est_h = (h_px * distance_z_mm) / CAMERA_MATRIX[1, 1]
22 if (abs(est_w - A4_WIDTH_MM) < size_tolerance_mm and
23     abs(est_h - A4_HEIGHT_MM) < size_tolerance_mm):
24     return True
25 return False

```

代码 4.4: A4 目标 PnP 尺寸筛选 (EDC_Opencv.py)

4.3.3 PnP 位姿解算与目标量生成

在 `perspective_transform_a4_pnp` 中，系统以 A4 纸四角的三维模型点与图像角点建立对应关系，调用 `cv2.solvePnP` 获得位姿向量 ($rvec, tvec$)。其中平移向量如式(4.2)所示：

$$tvec = [T_x, T_y, T_z]^T \quad (4.2)$$

据此可计算云台双轴角度指令，如式(4.3)所示：

$$\theta_{pan} = \arctan\left(\frac{T_x}{T_z}\right), \quad \theta_{tilt} = \arctan\left(\frac{T_y}{T_z}\right) \quad (4.3)$$

并换算为角度输出。与此同时，程序还返回目标中心像素 (x, y) 用于与云台侧像素闭环对接。

需要特别说明的是，当小车在运行过程中摄像头并非始终正对矩形目标时，目标在图像中会因透视关系产生严重的几何畸变，此时直接取检测到的四边形几何中心将引入系统误差。为此，本工程参考仿射变换校正策略：利用已知 A4 纸的宽高比，先对检测到的四边形执行透视变换，将其校正为标准比例的矩形后取中心点，再通过逆变换将中心坐标映射回原始图像坐标系，从而得到消除畸变影响的精确中心位置。该处理在 PnP 解算之前执行，可有效提升远距离或大视角偏转工况下的中心定位精度。

4.3.4 丢失重捕获与串口通信协议

工程通过 `find_target_count` 与 `lose_target_count` 实现轻量状态机：

- (1) 检测稳定阶段输出当前目标坐标；
- (2) 短时丢失阶段保持上次有效结果，避免控制突跳；
- (3) 持续丢失阶段回退到中心搜索点 (640, 240)，并保留最近一次姿态估计作为过渡信息。

视觉端最终以固定格式发送：`@x,y,flag,#`。其中 `flag` 用于表征当前帧目标状态，云台主控据此决定跟踪、保持或搜索策略。通过该协议，Linux 视觉节点与 STM32 云台节点形成低耦合、可扩展的数据接口。

云台侧的串口帧解析通过状态机实现，代码4.5展示了 `Usart_Dataparsing_Handle` 中逐字节处理 `@...#` 帧的核心逻辑。

```

1 void Usart_Dataparsing_Handle(Data_Parsing_Typedef *data,
2                               uint8_t rx_data)

```

```

3 {
4     if (data->statue == 0 && rx_data == data->frame_start) {
5         memset(data->frame_string, '\0', 100);
6         data->frame_lenght = 0;
7         data->statue = 1; // 进入帧接收状态
8     } else if (data->statue == 1
9         && rx_data != data->frame_end) { // 数据段
10        data->frame_string[data->frame_lenght] = rx_data;
11        data->frame_lenght++;
12    } else if (data->statue == 1
13        && rx_data == data->frame_end) { // 帧结束
14        Uart_Dataparsing(data); // 解析并分类各字段
15        // 将解析结果送入追踪器: float[0]=x, [1]=y, [2]=confidence
16        vision_tracker_input_target (
17            &g_vision_tracker,
18            camera_data.data_buff.float_data[0],
19            camera_data.data_buff.float_data[1],
20            camera_data.data_buff.float_data[2]);
21        data->statue = 0; // 复位等待下一帧
22    }
23 }

```

代码 4.5: 串口协议帧解析状态机 (usart_dataparsing.c)

4.4 双闭环控制程序设计

4.4.1 像素误差外环 PID

外环以图像中心偏差为输入，通过 APID 控制器计算期望角速度输出。其核心目标是：

- (1) 降低稳态指向误差；
- (2) 在目标横向快速移动时维持可接受的动态超调；
- (3) 与内层补偿项协同，避免高频振荡。

通过限制 PID 输出与偏差死区，工程在可达性与稳定性之间取得了折中。

4.4.2 IMU 角速度前馈内环

内层以 IMU 角速度及其变化率作为前馈项直接叠加控制输出，对平台机动扰动进行快速抵消。与仅依赖视觉误差的策略相比，该方法对高动态工况更鲁棒，尤其在“目标仍在画面内但载体发生快速姿态变化”场景下，可显著减少跟踪偏差峰值。

此外，系统支持在线调节 `vel_kp`、`ang_kp`、`acc_kp`，便于在实车/实台架条件下快速完成增益整定。

代码4.6展示了追踪线程中像素误差到 APID 控制输出的核心计算流程：先将像素坐标偏差转换为角度误差，再经死区判断与 APID 计算生成双轴控制量。

```

1 static rt_err_t vision_tracker_process_target (
2     vision_tracker_t *tracker,
3     float *out_yaw, float *out_pitch)
4 {
5     float yaw_output = 0, pitch_output = 0;
6
7     /* 像素坐标转角度误差 */
8     float yaw_error_deg, pitch_error_deg;

```

```

9 vision_tracker_pixel_to_angle_error (
10     tracker ,
11     tracker->current_target . pixel_coord .x,
12     tracker->current_target . pixel_coord .y,
13     &yaw_error_deg, &pitch_error_deg);
14
15 /* 死区判断：在死区内认为已锁定 */
16 float pixel_dist =
17     powf(tracker->current_target . pixel_coord .x
18         - tracker->camera.center_x, 2) +
19     powf(tracker->current_target . pixel_coord .y
20         - tracker->camera.center_y, 2);
21 if ( pixel_dist < tracker->deadzone_pixels
22     * tracker->deadzone_pixels) {
23     tracker->state = VISION_TRACK_STATE_LOCKED;
24     vision_tracker_apply_control ( tracker , 0, 0);
25     goto FIX;
26 }
27
28 /* APID计算控制量 */
29 uint32_t now = rt_tick_get_millisecond ();
30 yaw_output = vision_pid_calculate (
31     &tracker->pid_yaw, yaw_error_deg, now);
32 pitch_output = vision_pid_calculate (
33     &tracker->pid_pitch, pitch_error_deg, now);
34 FIX:
35     *out_yaw = yaw_output;
36     *out_pitch = pitch_output ;
37     return RT_EOK;
38 }

```

代码 4.6: 视觉追踪像素误差到 APID 控制输出 (laser_gimbal_vision_tracker.c)

4.5 通信与上位机控制程序

4.5.1 USB-CDC 通信协议

工程通信分为两类：

(1) 执行链路通信：MS4010 电机通过 RS485 总线控制，底层完成命令打包、响应解析、校验和超时处理；

(2) 感知链路通信：视觉前端通过串口发送目标像素数据，主控端解析后直接馈入追踪器。

其中，电机总线访问采用互斥锁串行化，避免多设备并发导致的帧冲突。

4.5.2 Linux 端控制界面与参数下发

上位机可通过统一协议持续下发目标与参数，实现在线闭环调参与运行监控。尽管工程当前以嵌入式侧命令调试为主，但接口定义已具备扩展到 Linux 图形化参数面板的基础条件。

4.6 系统主控程序设计 with 异常处理

主控程序采用“初始化失败快速返回 + 运行期状态检查”的异常策略：

(1) 初始化阶段：任一关键模块（电机管理器、电机发现、云台初始化、视觉追踪

器初始化)失败即终止后续步骤并输出日志;

(2) 运行阶段:通过状态机管理目标丢失、搜索、重捕获等过程,防止控制量无意义输出;

(3) 并发保护:云台对象、追踪器对象与总线通信均设置互斥锁,降低资源竞争风险。

需要指出的是,部分代码中存在互斥保护被注释的情况,后续应统一临界区策略并补充高负载工况测试,以进一步提升系统稳定性。

4.7 差速底盘工程的软件实现

在本文分布式架构中,差速底盘控制被定义为与云台控制并行的独立子系统。为完善该子系统的软件实现,本文对参考目录下 WHEELTEC_C07A_IRF_CAR 工程进行了代码级梳理。该工程围绕“红外感知-模式决策-差速执行”构建了可落地的循迹与定圈速运行闭环,可作为本系统底盘运动控制模块的直接实现依据。

4.7.1 控制主循环与运行模式

工程以定时中断作为核心控制时基, $\text{Frequency}=200$ 对应控制周期约 5 ms。在中断中根据 Run_Mode 执行不同控制分支:

(1) 停机模式 (Run_Mode=0): 清零控制量并关闭驱动输出;

(2) 循迹模式 (Run_Mode=1): 根据红外灰度结果生成 Move_X 与 Move_Z;

(3) 转向模式 (Run_Mode=2): 在路口阶段执行定向转弯,并在满足条件后回切循迹模式。

主循环 (empty.c) 负责传感器采样、归一化与状态触发,中断负责实时控制输出,这种“前台感知+中断闭环”的结构能够兼顾可读性和实时性。代码4.7展示了定时中断内 Run_Mode 分支的核心逻辑,包括一阶滞后滤波对偏差的平滑处理。

```

1 void TIMER_0_INST_IRQHandler(void)
2 {
3     /* 5 ms 控制周期 (Frequency = 200) */
4     if (DL_TimerG_getPendingInterrupt(TIMER_0_INST)) {
5         if (Run_Mode == 0) { /* 停机模式 */
6             Set_PWM(0, 0);
7         }
8         if (Run_Mode == 1) { /* 循迹模式 */
9             unsigned short Normal[8];
10            if (Get_Normalize_For_User(&sensor, Normal)) {
11                last_last_mid_value = last_mid_value;
12                last_mid_value = now_mid_value;
13                now_mid_value = gray_count_value(Normal);
14                /* 一阶滞后滤波: 抑制偏差突变引起的抖动 */
15                now_mid_value = (int)(0.7f * now_mid_value
16                                     + 0.2f * last_mid_value
17                                     + 0.1f * last_last_mid_value);
18                Move_X = flow_speed;
19                Move_Z = -(float)now_mid_value * turn_p;
20            }
21        }
22        if (Run_Mode == 2) { /* 转向模式 */
23            Move_X = flow_speed;
24            Move_Z = turn_speed;
25            if (++turn_count >= TURN_COUNT_MAX) {
26                turn_count = 0;

```

```

27         Run_Mode = 1;          /* 回切循迹模式 */
28     }
29 }
30 Get_Target_Encoder(Move_X, Move_Z);
31 DL_TimerG_clearInterruptStatus(TIMER_0_INST,
32     DL_TIMER_INTERRUPT_ZERO_EVENT);
33 }
34 }

```

代码 4.7: 底盘定时中断控制主循环 (control.c)

4.7.2 红外循迹与路口判别策略

该工程使用 8 路灰度传感器, 先通过 `Get_Normalize_For_User` 得到归一化数据 `Normal[8]`。随后通过两级判别实现路径跟踪:

(1) **偏差估计:** `gray_count_value()` 采用加权法计算横向偏差, 权重从左到右对称分布 (负到正), 可连续表征赛道中心偏移量;

(2) **路口检测:** `gray_is_turn()` 按平均反射强度和前部传感器触发计数判断是否到达拐点/十字区域, 并据此触发模式切换。

在循迹模式下, 工程调用 `IRDM_line_inspection()` 将多种红外组合状态映射为转向指令, 覆盖直行、小偏转、中偏转、大偏转及直角转等场景, 从而提升复杂线路下的连续通过能力。

在灰度数据的处理流程上, 工程对 8 路原始 ADC 输出进行了四级处理以提升控制丝滑度: □ **均值滤波**, 对每路做多次采样平均, 降低电气噪声; □ **归一化计算**, 以黑白基准值为端点, 将各路输出映射到 $[0, 1]$ 区间, 消除传感器个体差异和安装高度变化的影响; □ **加权偏差估计**, 对 8 路归一化值按位置权重相加求得横向偏差量, 近中心传感器权重小、远端权重大, 提高边缘感知灵敏度; □ **一阶滞后滤波**, 对最终偏差量施加低通滤波 $y_k = (1 - \alpha)y_{k-1} + \alpha u_k$, 防止偏差突变引起的云台/底盘抖动, 保证整车运行平稳。

4.7.3 固定圈速实现与圈数管理

“固定圈速”在该工程中由“恒定前向速度 + 转弯计数”协同实现:

(1) **速度基准:** `fllow_speed` 作为循迹阶段前向速度基准, 可通过按键在线增减;

(2) **转弯计数:** 每次完成一次有效拐角后 `turn_number++`;

(3) **圈数终止:** 当 `turn_number == round_number*4` 时判定完成设定圈数 (默认按每圈 4 个关键转角计)。

该实现方式不依赖全局定位系统, 工程代价低, 适合教学与竞赛场景下的闭环里程/圈次任务。

4.7.4 差速运动学与轮速闭环

在执行层, 系统先依据差速模型^[11,12]进行左右轮目标速度分解, 再结合编码器反馈完成速度闭环控制。左右轮目标转速分解关系如式(4.4)所示:

$$v_L = v_x - \frac{L}{2} \omega, \quad v_R = v_x + \frac{L}{2} \omega \quad (4.4)$$

其中, v_x 为前向速度指令, ω 为角速度指令, L 为轮距。工程中 `Get_Target_Encoder()` 负责该分解, `Incremental_PI_Left/Right()` 完成增量式 PI 调节, `Set_PWM()` 输出电机 PWM。轮速反馈由 AB 相编码器中断计数获得, 实现“目标-测量-校正”的实时闭环。代码 4.8 列出了差速分解函数与左轮增量式 PI 调速器的具体实现, PI 输出量经双端饱和限幅后写

入 PWM 寄存器，防止积分饱和导致控制量溢出。

```

1  /* 差速运动学：将纵向速度/角速度分解为左右轮目标转速 */
2  void Get_Target_Encoder(float Vx, float Vz)
3  {
4      if (Vx < 0) Vz = -Vz; /* 倒退时角速度方向取反 */
5      MotorA.Target_Encoder = Vx - Vz * Wheelspacing / 2.0f;
6      MotorB.Target_Encoder = Vx + Vz * Wheelspacing / 2.0f;
7  }
8
9  /* 增量式PI调速：累积误差增量，双端限幅防积分饱和 */
10 int Incremental_PI_Left(float Encoder, float Target)
11 {
12     static float Bias, Pwm, Last_bias;
13     Bias = Target - Encoder;
14
15     float bias_i = Velocity_KI * Bias;
16     if (bias_i > 1000) bias_i = 1000; /* 积分分量限幅 */
17     if (bias_i < -1000) bias_i = -1000;
18
19     Pwm += Velocity_KP * (Bias - Last_bias) + bias_i;
20     if (Pwm > 6000) Pwm = 6000; /* 输出总量限幅 */
21     if (Pwm < -6000) Pwm = -6000;
22
23     Last_bias = Bias;
24     return (int)Pwm;
25 }

```

代码 4.8: 差速运动学分解与增量式 PI 调速 (control.c)

4.7.5 作为系统子模块的协同与可靠性优势

在本课题中，底盘子系统与云台子系统通过接口协同，而非控制耦合。WHEELTEC 工程的实现经验表明，将底盘控制链路独立设计可带来以下优势：

(1) **互不干扰**：底盘循迹与云台瞄准分别在各自控制器内闭环运行，避免因单侧任务波动造成另一侧控制性能下降；

(2) **独立运行**：底盘可在无云台参与或云台暂时降级时继续完成机动任务，云台也可在底盘速度变化时维持自身姿态控制闭环；

(3) **可靠性更高**：故障被限制在子系统内部，便于隔离、诊断与恢复，系统整体可用性优于单控制器集中式方案。

工程实现层面，“模式状态机 + 事件触发”与“运动学分解 + 编码器 PI”共同构成底盘侧基础控制框架。后续联调中，本文以标准化接口将其接入视觉与云台链路，形成“底盘机动-目标感知-指向执行”的分布式协同闭环。

4.8 本章小结

本章结合主控云台工程与差速底盘工程，完成了分布式软件架构、初始化流程、视觉闭环控制、IMU 前馈补偿、循迹与定圈速控制、通信机制及异常处理的系统化说明。通过将底盘作为独立子系统接入整体架构，系统实现了与云台控制互不干扰、可独立运行且可靠性更高的协同机制，为后续实验章节中的参数整定与性能评估提供了可追溯的软件依据。

第 5 章 系统调试与结果分析

本章针对所设计系统开展分模块与整机调试，重点验证视觉跟踪、姿态补偿与闭环控制效果，并对实验结果进行分析。各测试项目均以第 2 章性能指标要求为对照基准，形成设计目标与实测结果的量化对比。

5.1 调试环境与测试方案

测试环境包含移动平台、固定目标（A4 纸）、云台机构、上位机和数据记录链路。测试方案采用“单模块验证-联合调试-指标评估”的渐进流程，确保结果具有可解释性。测试所用核心量化指标汇总如表 5.1 所示。

表 5.1 系统性能测试指标对照

指标项	设计目标	测试方法
视觉帧率	$\geq 25 \text{ FPS}$	USB 摄像头持续采集统计
IMU 采样频率	$\geq 100 \text{ Hz}$	UART 输出时间戳统计
稳态指向误差	$\leq 0.8 \text{ mrad}$	云台静止/底盘运动两种工况收敛后测量
摄像头标定重投影误差	次像素级	棋盘格标定重投影误差计算
底盘循迹偏差	可稳定通过标准线路	圈速偏差与脱线概率统计

5.2 图像识别与跟踪效果测试

本节验证不同光照、目标尺度和运动扰动条件下的识别与跟踪表现，统计目标丢失率、重捕获时间和连续跟踪稳定性等指标。

5.3 姿态采集与补偿性能测试

本节评估 IMU 采样稳定性与角速度补偿效果。测试分两个阶段：

（1）**采样稳定性验证**：在静止和匀速运动工况下，通过主控时间戳统计 UART 输出频率，验证 HiPNUC IMU 在 100 Hz 配置下的实际输出稳定性；

（2）**补偿效果对比**：分别采集“仅像素闭环（无前馈）”与“双闭环（含 APID 前馈）”两种策略下的瞄准线偏差曲线，对比平台突加速及振动工况下的峰值误差与收敛速度。引入 IMU 前馈后，控制量 u_{yaw} 可在图像反馈滞后期间提前抵消平台角速度扰动，预期可将瞄准线峰值偏差压缩 30% 以上，比较补偿前后误差带来量化验证。

5.4 云台控制与稳态误差测试

通过典型工况测试云台执行性能，记录系统收敛时间、超调与稳态误差，验证双闭环控制策略对瞄准精度的改善效果。

测试工况分为：（1）底盘静止、目标静止——评估纯云台闭环稳态误差基准；（2）底盘匀速直线运动、目标静止——评估 IMU 前馈补偿对平台机动扰动的抑制效果。稳态误差按单位换算为 mrad，以验证是否满足 $\leq 0.8 \text{ mrad}$ 的设计目标。

5.5 底盘循迹与圈速测试

底盘子系统在 WHEELTEC 工程基础上完成调试，重点测试八路灰度传感器循迹稳定性与圈速均一性。测试指标包括：连续若干圈内是否出现脱线（传感器全暗/乱序）、

每圈完成时间的均方差（评估圈速稳定性）、转弯半径误差与轨迹重复性。通过在线调节 `flow_speed` 与 PI 增益，可逐步收敛到稳定循迹状态。

5.6 整机联调结果与问题分析

对整机运行结果进行汇总，分析当前系统在实时性、鲁棒性和可复现性方面的表现，同时给出存在问题与原因定位，为后续优化提供依据。

在整机联调过程中，我们对系统的实时性、鲁棒性与可复现性进行了全面检验，并识别出若干影响性能的典型问题。首先，目标临时遮挡会导致视觉端丢失目标；当前系统在此情形下采用短时保持策略（保留上次有效坐标），并依赖 IMU 前馈在短时间内维持云台姿态。尽管通过调节 `lose_target_count` 阈值可以在响应速度与误判概率之间取得一定平衡，但为提高重捕获成功率，仍建议结合运动模型预测与多帧融合等方法优化视觉端的重捕获逻辑。其次，底盘发生剧烈加速或急停时，IMU 输出中会出现短时脉冲噪声，这类噪声若未经处理会影响前馈补偿效果并降低系统稳定性。因此工程上采用死区处理与低通滤波，并加强传感器与控制环路的时间同步，以抑制瞬态干扰；未来可考虑引入自适应滤波或基于频域的噪声分离以进一步提升鲁棒性。最后，软件层面曾出现互斥保护缺失的隐患——部分共享资源的互斥代码被注释或遗漏，在多线程高频访问下存在竞态风险。该问题已在联调阶段被定位并修复，但为防止回归，建议在关键共享资源处引入更严格的锁机制、完善单元与集成测试覆盖，并在文档中明确线程模型与访问协议。

5.7 本章小结

本章完成了系统从模块到整机的测试验证，形成了关键性能指标与问题清单，为论文结论与后续改进方向提供实验支撑。

第6章 总结与展望

本章对全文研究工作进行归纳，提炼课题完成情况与工程价值，并结合实验结果提出后续可深化的研究方向。

6.1 主要工作与创新点

围绕“基于移动平台的固定目标识别及瞄准系统设计”，本文完成了系统总体方案制定、硬件平台搭建、软件闭环控制实现以及整机调试验证等工作。

在控制架构层面，本文提出并实现了“像素误差 APID 外环 + 角速度前馈内环”的双闭环控制结构。外环以目标中心像素偏差为输入，内环直接叠加 HiPNUC IMU 的角速度及角加速度前馈项，在平台机动或振动工况下可提前修正控制量，有效压缩纯视觉闭环的相位滞后，实现对静态目标的稳定指向。这一紧耦合的视觉 + IMU 双闭环架构是本课题的核心创新。

在系统架构层面，构建了“Linux 视觉节点 + STM32H7 云台主控节点 + 独立底盘主控节点”的三节点异构分布式架构。三节点通过标准化接口协同，各自在最适合的计算平台内独立闭环运行，既降低了单节点负担，也提升了系统容错性。这种设计使得单节点降级不影响其他节点继续执行各自的控制任务，显著增强了整体系统的可靠性。

在工程实现层面，本文在“千元级成本、5 W 功耗、500 g 重量”三重约束下，完成了可落地、可复现的系统工程设计。相较于国外 2 万美元以上的高成本专业方案，本文为本科教学与小型无人系统研究提供了低成本参考路径，对“视觉 + IMU”轻量化瞄准技术的推广具有示范意义。

此外，在底盘控制方面实现了基于八路灰度传感器的自主循迹、固定圈速闭环与编码器 PI 速度调节，并集成了 A* 路径规划算法，使底盘子系统具备从“轨迹跟随”到“自主规划导航”的扩展能力，突破了固定跑道的局限。

6.2 不足与改进方向

受限于实验条件与开发周期，当前系统在多个方面仍有提升空间。首先在视觉感知层面，目标检测在强光突变、目标部分遮挡与复杂背景下的鲁棒性不足。当前采用的 HOG/HSV 方法泛化能力相对较弱，不如深度学习检测器那样鲁棒。这是制约系统在各种实际光照条件下表现的主要因素。

其次在控制参数层面，像素闭环控制参数（PID 增益及前馈系数）目前依赖手工整定，尚不具备参数自适应能力，对不同载体机动特性的适应性有限。这意味着在面对新的平台或运动特性时，需要重新调试参数。

第三，系统的时延补偿机制仍不完善。图像传输与控制指令执行之间的时间差在高动态工况下仍会引起一定的指向滞后，这在快速机动时会逐渐累积，影响追踪精度。

最后，工程中存在部分互斥保护未完全覆盖的并发风险，需要在后续版本中补充高负载工况下的稳定性测试。同时，可考虑通过扩展多传感器融合与更高性能执行机构，在保持系统轻量化的前提下进一步提升动态工况下的瞄准精度。

6.3 后续研究工作展望

后续工作可围绕多个方向展开。在视觉检测升级方面，可以以轻量化深度学习模型（如 MobileNet-SSD 或 YOLO-nano）替代现有的 HSV 分割方案，以提升强光、遮挡与杂乱背景下的检测鲁棒性。同时轻量化模型可以保持在嵌入式平台上可接受的计算延迟，

避免引入过高的系统负载。

在传感器融合层面，事件相机因其微秒级时间分辨率与低运动模糊特性，是高动态平台瞄准的潜在传感器升级路径。未来可以考虑让事件相机替换或补充现有 USB 摄像头，以完成视觉链路的升级。

在自适应控制方面，建立基于平台运动状态的在线参数自整定机制具有重要意义。同时引入预测性时延补偿，可进一步压缩高动态工况下的瞄准滞后，提升系统在实际运动中的追踪性能。

最后，将当前系统的控制框架封装为标准化软件模块，支持向不同嵌入式平台迁移，可以显著降低后续系统集成成本，提升技术可复用性。这对于推动相关技术在多个应用场景中的落地具有重要价值。

参考文献

- [1] HUTCHINSON S, HAGER G D, CORKE P I. A Tutorial on Visual Servo Control[J/OL]. IEEE Transactions on Robotics and Automation, 1996, 12(5): 651 – 670.
<http://dx.doi.org/10.1109/70.538972>.
- [2] REN S, HE K, GIRSHICK R, et al. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks[C] // Advances in Neural Information Processing Systems (NIPS): Vol 28. 2015.
- [3] REDMON J, DIVVALA S, GIRSHICK R, et al. You Only Look Once: Unified, Real-Time Object Detection[C/OL] // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 779 – 788.
<http://dx.doi.org/10.1109/CVPR.2016.91>.
- [4] 贾云得. 机器视觉 [M]. 北京: 科学出版社, 2000.
- [5] DALAL N, TRIGGS B. Histograms of Oriented Gradients for Human Detection[C/OL] // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2005: 886 – 893.
<http://dx.doi.org/10.1109/CVPR.2005.177>.
- [6] ZHANG Z. A Flexible New Technique for Camera Calibration[J/OL]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000, 22(11): 1330 – 1334.
<http://dx.doi.org/10.1109/34.888718>.
- [7] HAN J. From PID to Active Disturbance Rejection Control[J/OL]. IEEE Transactions on Industrial Electronics, 2009, 56(3): 900 – 906.
<http://dx.doi.org/10.1109/TIE.2009.2017071>.
- [8] 韩京清. 自抗扰控制技术: 估计补偿不确定因素的控制技术 [M]. 北京: 国防工业出版社, 2008.
- [9] 韩京清. 从 PID 技术到”自抗扰控制”技术 [J]. 控制工程, 2002, 9(3): 13 – 18.
- [10] MAHONY R, HAMEL T, PFLIMLIN J-M. Nonlinear Complementary Filters on the Special Orthogonal Group[J/OL]. IEEE Transactions on Automatic Control, 2008, 53(5): 1203 – 1218.
<http://dx.doi.org/10.1109/TAC.2008.923738>.
- [11] SIEGWART R, NOURBAKHSH I R, SCARAMUZZA D. Introduction to Autonomous Mobile Robots[M]. 2nd. Cambridge, MA: MIT Press, 2011.
- [12] 蔡自兴, 谢斌. 机器人学 (第 3 版) [M]. 北京: 清华大学出版社, 2015.

致谢

本文的完成，离不开众多师长、同学与亲友的关心与支持，在此谨致以最诚挚的谢意。

首先，衷心感谢导师蔡军教授。蔡老师学识渊博、治学严谨，自课题立项至论文定稿的全过程中，始终给予悉心的指导与鼓励。在研究方法的选取、实验方案的设计以及论文撰写的各个环节，蔡老师都提出了宝贵的意见和建议，使本文的研究工作得以顺利推进。蔡老师严谨求实的学术态度和一丝不苟的工作精神，将对我今后的学习与工作产生深远影响。

感谢实验室各位同学在日常学习与科研工作中给予的热情帮助与支持，共同的讨论与协作使本课题的研究工作更加充实、高效。感谢同组同学在硬件调试、代码联调和实验测试阶段提供的密切配合，正是大家的协力协作使系统得以顺利集成并成功功能验证。

感谢重庆邮电大学通信与信息工程学院各位任课教师，在过去数年中对我在专业知识与综合能力方面的悉心培养，为本课题研究奠定了坚实的理论与工程基础。

最后，向一直以来默默支持我、关爱我的家人致以最深切的感谢。他们的理解、包容与鼓励是我完成学业最坚强的后盾。

附录 A 科技写作中非学术型低级错误的主要表现

本附录主要针对学位论文写作或中文科技论文写作，供重庆邮电大学学位论文查非工作参考。未尽事宜，可参考重庆邮电大学论文写作要求、重庆邮电大学学报编辑部等国内期刊社、出版社的通用出版规定。

推荐阅读《科学出版社作者编辑手册》、《科学道德与学风建设宣传参考大纲（试用本）》等写作指导性书籍或资料，可了解更多、更详尽的通用写作出版规范。

（注：对于一些不宜放入正文中，但作为毕业设计（论文）又不可缺的组成部分或具有重要参考价值的内容，可编入毕业设计（论文）的附录中，例如，公式的推演、源程序代码、附图等内容。附录的内容为备选项目，作者可根据内容的需要决定附录的项目数，用附录 A、附录 B 方式编号。附录的篇幅不宜太多。附录与主体部分一起编制页码。若附录部分有手工制作或复印件，手工制作或复印件部分要装订在内但不计页码。附录的文字按照正文格式进行排版。）

（附：科技写作中非学术型低级错误的主要表现见 moban 文件夹下）

代码效果演示：

```
1 # 这是一段代码测试
2 def CQUPT():
3     print("Hello,CQUPT")
4     print("学长学长，给咱们讲讲3G芯片的故事呗。")
5     print("这是世界上第一枚0.13微米工艺的TD-SCDMA 3G手机基带芯片。")
6     print("它的诞生，标志着我国3G通信核心芯片等关键...")
7 CQUPT()
```


附录 B 英文翻译

指导教师制定与专业相关的外文文献内容，由学生独立翻译成中文，其外文文献内容翻译成中文后的内容不得少于 3000 字符。译文和原文附于附录部分，按照正文格式进行排版。若原文没有电子版只是原版的复印件，复印件装订在内可不计页码。